

STAT 240

Week 5: More SQL and Linear Regression

Owen G. Ward

This week

- Recap SQL from last day
 - Getting data out of a SQL database
- First key data science tool
 - Regression
 - Brief introduction to Optimization

Midterm exam

- In class midterm exam Feb 25th. It is to be done individually
 - Exam will be held in person in AQ3181
 - You must bring your laptop to complete the exam. I will provide you with a template `qmd` file in advance, make sure you can Render it
-
- You can use the class notes and examples during the exam, R help files, but cannot use any web resources.
 - You can only use the internet to upload to crowdmark when submitting
 - **You cannot use any AI tools or communicate with anyone in/outside the exam. Doing so is a breach of academic integrity**
 - If you have trouble rendering during the exam make sure to use the tips we discussed last week.
 - **Failure to submit a pdf will be an automatic 0!**
 - I'll post a (short) practice exam this week, discuss briefly in class next Friday

Databases: Review

Databases are systems that let us manipulate data in a way that's synchronous, secure and efficient

- SQL is an example of a database language
- MySQL is a specific implementation of SQL. (You can't install SQL. You can, however, install MySQL)
- SQLite is another implementation of SQL (it's easier to use)

Queries from Last Week

- We saw some basic queries last time

```
library(RSQLite)
db <- dbConnect(SQLite(), dbname = "shapes_week4.sqlite")
dbListTables(db)
```

```
[1] "Diamonds" "Platonic"
```

```
request_new <- dbSendQuery(
  db,
  "SELECT * FROM Platonic WHERE Sides > 5"
)
dbFetch(request_new, n = 1)
```

```
  Name Sides
1 Cube      6
```

```
dbClearResult(request_new)
```

Ordering Data

- Use `ORDER BY` to get the raw data ordered already
- This could be useful for investigating the max/min

```
request_order <- dbSendQuery(
  db,
  "SELECT * FROM Diamonds ORDER BY carat"
)
head(dbFetch(request_order))
```

	carat	cut	color	clarity	depth	table	price	x	y	z
1	0.2	Premium	E	SI2	60.2	62	345	3.79	3.75	2.27
2	0.2	Premium	E	VS2	59.8	62	367	3.79	3.77	2.26
3	0.2	Premium	E	VS2	59.0	60	367	3.81	3.78	2.24
4	0.2	Premium	E	VS2	61.1	59	367	3.81	3.78	2.32
5	0.2	Premium	E	VS2	59.7	62	367	3.84	3.80	2.28
6	0.2	Ideal	E	VS2	59.7	55	367	3.86	3.84	2.30

```
dbClearResult(request_order)
```

Largest First

- Just add a DESC at the end of the query

```
request_order <- dbSendQuery(
  db,
  "SELECT * FROM Diamonds ORDER BY carat DESC"
)
head(dbFetch(request_order))
```

	carat	cut	color	clarity	depth	table	price	x	y	z
1	5.01	Fair	J	I1	65.5	59	18018	10.74	10.54	6.98
2	4.50	Fair	J	I1	65.8	58	18531	10.23	10.16	6.72
3	4.13	Fair	H	I1	64.8	61	17329	10.00	9.85	6.43
4	4.01	Premium	I	I1	61.0	61	15223	10.14	10.10	6.17
5	4.01	Premium	J	I1	62.5	62	15223	10.02	9.94	6.24
6	4.00	Very Good	I	I1	63.3	58	15984	10.01	9.94	6.31

```
dbClearResult(request_order)
```

Summarising across groups

- A key data science skill is constructing summaries of a table of data across a specific categorical variable
- If you took 260, you would have seen this with `group_by` and `summarise`
- There are very similar options in SQL

Computing Means

```
request_avg <- dbSendQuery(  
  db,  
  "SELECT cut, COUNT(*) AS n, AVG(price) AS avg_price  
  FROM diamonds  
  GROUP BY cut"  
)  
head(dbFetch(request_avg))
```

	cut	n	avg_price
1	Fair	1610	4358.758
2	Good	4906	3928.864
3	Ideal	21551	3457.542
4	Premium	13791	4584.258
5	Very Good	12082	3981.760

```
dbClearResult(request_avg)
```

- Note the ordering is not the order of operations

Databases: SELECT and INSERT

- `SELECT * FROM Name` returns all columns and rows from the table called Name
- `SELECT * FROM Name WHERE ...` returns all columns from the table called Name for rows in which the conditions listed in ... are true

```
request_new <- dbSendQuery(  
  db,  
  "SELECT * FROM Platonic WHERE Sides > 5"  
)
```

-
- `SELECT V1, V2 FROM Name` returns columns V1 and V2 of all rows in the table called Name
 - `INSERT INTO Name VALUES (v1, v2, ...)` Adds the row given by the vector *v1*, *v2*, ... into the table called Name

```
request <- dbSendQuery(
  db,
  "SELECT carat, cut FROM Diamonds WHERE price < 330 AND depth > 60"
)
dbFetch(request)
```

```
  carat  cut
1 0.23 Ideal
```

```
dbClearResult(request)
```

```
request <- dbSendQuery(
  db,
  "SELECT * FROM Diamonds WHERE price < 330 OR depth > 60"
)
head(dbFetch(request))
```

```
  carat    cut color clarity depth table price    x    y    z
1 0.23   Ideal    E   SI2   61.5    55   326 3.95 3.98 2.43
2 0.21 Premium    E   SI1   59.8    61   326 3.89 3.84 2.31
3 0.23    Good    E   VS1   56.9    65   327 4.05 4.07 2.31
4 0.29 Premium    I   VS2   62.4    58   334 4.20 4.23 2.63
5 0.31    Good    J   SI2   63.3    58   335 4.34 4.35 2.75
6 0.24 Very Good    J  VVS2   62.8    57   336 3.94 3.96 2.48
```

```
dbClearResult(request)
```

Notes on SQL

- For comparison to non numeric data, use 'Single quotes' along with single = for comparison

```
request <- dbSendQuery(  
  db,  
  "SELECT *  
  FROM diamonds  
  WHERE clarity = 'SI2' "  
)  
head(dbFetch(request))
```

	carat	cut	color	clarity	depth	table	price	x	y	z
1	0.23	Ideal	E	SI2	61.5	55	326	3.95	3.98	2.43
2	0.31	Good	J	SI2	63.3	58	335	4.34	4.35	2.75
3	0.31	Ideal	J	SI2	62.2	54	344	4.35	4.37	2.71
4	0.20	Premium	E	SI2	60.2	62	345	3.79	3.75	2.27
5	0.30	Ideal	I	SI2	62.0	54	348	4.31	4.34	2.68
6	0.30	Good	I	SI2	63.3	56	351	4.26	4.30	2.71

```
dbClearResult(request)
```

Exercise

- Write a SQL request to compute the average carats in Premium cut diamonds, across different levels of color. For each level also compute how many rows there are

Exercise Solution

```
request <- dbSendQuery(  
  db,  
  "SELECT  
  color,  
  COUNT(*) AS n,  
  AVG(carat) AS avg_carat  
  FROM diamonds  
  WHERE cut = 'Premium'  
  GROUP BY color;"
```

```
)  
head(dbFetch(request), n = 7) ## only 7 colors here
```

	color	n	avg_carat
1	D	1603	0.7215471
2	E	2337	0.7177450
3	F	2331	0.8270356
4	G	2924	0.8414877
5	H	2360	1.0164492
6	I	1428	1.1449370
7	J	808	1.2930941

```
dbClearResult(request)
```

-
- Just saw GROUP BY and summarizing across this grouping
 - Many other options available, may be of use also
 - If you’ve seen joins between two data tables, these can be done directly in SQL
 - The SQLite manual is [here](#)

Case study: Working for Z Co Retail

Imagine you land a job as a data scientist at Z Co Retail, a small chain of department stores.

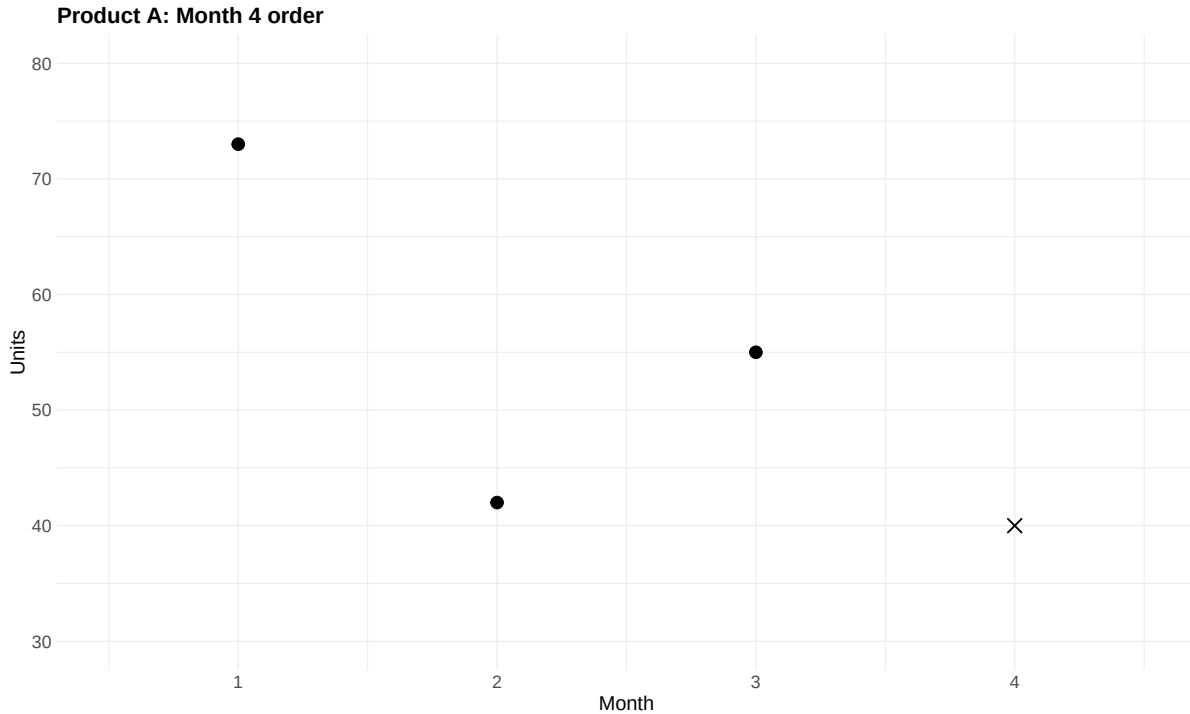
- Your first job is to automate and improve their process for inventory acquisition.
- They put out orders for inventory at the last business day of each month.

Here’s their current process:

- “Usually, each product sells about the same number of units from one month to the next”
- “So, for most products, we look at how many units it sold in the previous month. Then, we add 10% and subtract the number of units we still have in stock. That gives us the number of units to order”

Case study

- “for some products there are trends from month-to-month, we look at the previous three months and sort of eye-ball it”



Case study

- On your first day, your boss asks you if you can improve their ad-hoc method of penciling a cross onto a printout of a plot.
- They also ask you if you can automate the extrapolation, so they can do this more advanced method on a greater range of products
- This is a simple but important problem!

Solving this problem

- There is a broad literature of statistical methods that would work here (including the most simple approach: linear regression). And there is so much room to improve upon their standard operating procedure (SOP)

- But then your boss gives you the keys to their MySQL database (in these lecture notes, the mock database is *xcoretail.sqlite*), and then heads to another meeting.
- This SQLite database contains real data from an online retailer spanning the date range 2010-12-01 to 2011-12-09

Advanced SQL: Exploring a new database

- We'll go through some commands to explore a SQLite database (you can't `SELECT *` `FROM` a table if you don't know the table names)
- Here's how to find the names of the tables in a database (again):

```
library(RSQLite)
db <- dbConnect(SQLite(), dbname = "xcoretail.sqlite")
dbListTables(db)
```

```
[1] "prices" "sales"
```

-
- There are two tables: *sales* and *prices*. Let's print the first element from each

```
request <- dbSendQuery(db, "SELECT * FROM sales")
dbFetch(request, n = 1)
```

```
StockCode Quantity InvoiceDate
1      85123A          6 2010-12-01
```

```
dbClearResult(request)
request <- dbSendQuery(db, "SELECT * FROM prices")
dbFetch(request, n = 1)
```

```
StockCode          Description UnitPrice
1      85123A WHITE HANGING HEART T-LIGHT HOLDER      2.55
```

```
dbClearResult(request)
```

Advanced SQL: Exploring a new database

- Here's how to find the column names and types of a table in SQLite (without examining the result of a SELECT command):

```
request <- dbSendQuery(db, "pragma table_info('sales')")
dbFetch(request)
```

	cid	name	type	notnull	dflt_value	pk
1	0	StockCode	TEXT	0	NA	0
2	1	Quantity	INTEGER	0	NA	0
3	2	InvoiceDate	TEXT	0	NA	0

```
dbClearResult(request)
```

-
- This syntax is different from MySQL.
 - It's unlikely that the databases you end up using professionally will be exactly the same ones you study in school.
 - It's important to be able to read manuals to figure out how to do things in a database implementation that you haven't worked with before.
 - For example, [here's the relevant page](#) of the MySQL manual

Conditioning using LIKE

- We now know the table and column names of *xcoretail.sqlite*. Suppose we want to recreate the “printout” for *StockCode* “71053”, where month is 2011-05
- *LIKE* is a conditional operator, and it matches string values with a wild card % (so, 2011-05% matches 2011-05-01 but not 2011-04-01)

```
request <- dbSendQuery(db, "SELECT * FROM sales
                           WHERE StockCode = '71053' AND
                           InvoiceDate LIKE '2011-05%'" )
```

```
request <- dbSendQuery(db, "SELECT * FROM sales
                           WHERE StockCode = '71053' AND
                           InvoiceDate LIKE '2011-05%')

data <- dbFetch(request)
dbClearResult(request)
dim(data)
```

```
[1] 40  3
```

Data exploration

- Now that we have retrieved “71053” for 2011-05, we can do some data exploration to make sure our understanding of the data is correct

```
unique(data$StockCode)
```

```
[1] "71053"
```

```
unique(data$InvoiceDate)
```

```
[1] "2011-05-04" "2011-05-05" "2011-05-06" "2011-05-08" "2011-05-09"
[6] "2011-05-10" "2011-05-11" "2011-05-12" "2011-05-16" "2011-05-17"
[11] "2011-05-18" "2011-05-20" "2011-05-22" "2011-05-23" "2011-05-24"
[16] "2011-05-25"
```

Data exploration

```
min(data$Quantity)
```

```
[1] -1
```

```
max(data$Quantity)
```

```
[1] 120
```

```
sum(data$Quantity)
```

```
[1] 300
```

- Note that the minimum Quantity is negative. This is interesting, might be a refund? (Something to bring up at next meeting with our boss at Z Co Retail)

Making the “printout”

```
months <- c("2011-02", "2011-03", "2011-04", "2011-05")
counts <- rep(NA, 4)
for (i in 1:4) {
  query <- paste0("SELECT SUM(Quantity) FROM sales
                  WHERE StockCode = '71053' AND
                  InvoiceDate LIKE '", months[i], "%'")
  request <- dbSendQuery(db, query)
  counts[i] <- dbFetch(request)
  dbClearResult(request)
}
```

- Here we use `SELECT SUM( ... )` to perform an operation on a column, instead of selecting the column
- We use `paste0` to form the query string (this is a function that returns the string formed by concatenating its parameters)

Exercise

- Run the code on the previous slide
- Compare `counts` before you ran the code vs after it. Does the structure of it look the same?

A quick point

```
months <- c("2011-02", "2011-03", "2011-04", "2011-05")
i <- 1
query <- paste0("SELECT SUM(Quantity) FROM sales
                WHERE StockCode = '71053' AND
                InvoiceDate LIKE '", months[i], "%'")
request <- dbSendQuery(db, query)
str(dbFetch(request))
```

```
'data.frame':  1 obs. of  1 variable:
 $ SUM(Quantity): int 89
```

```
dbClearResult(request)
```

- Even though we only return a single number it is actually returning it in a data frame

-
- `str` is a function to display the structure of an object in R
 - A data frame is a list, so even though we created `counts` as a numeric vector...

```
counts
```

```
[[1]]
[1] 89

[[2]]
[1] 143

[[3]]
[1] 218

[[4]]
[1] 300
```

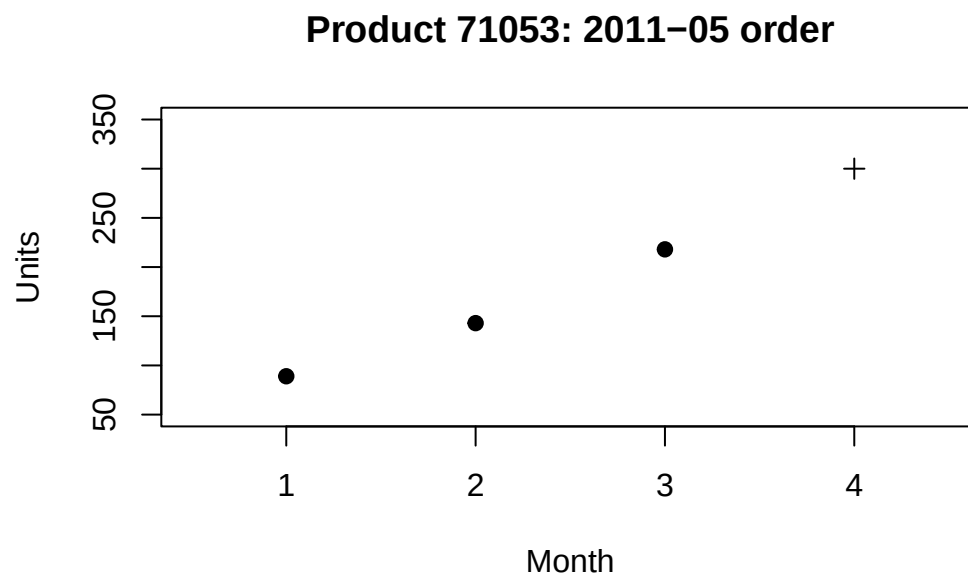
- Be wary of this with more complex queries

Creating the Plot

```
months
```

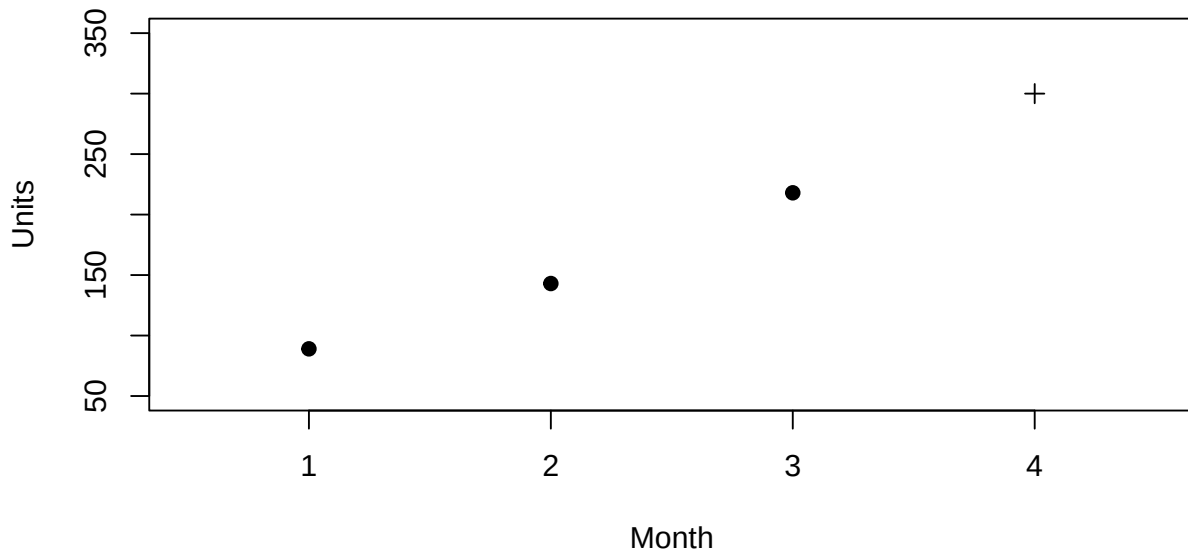
```
[1] "2011-02" "2011-03" "2011-04" "2011-05"
```

```
plot(c(1, 2, 3), counts[-4],  
     main = "Product 71053: 2011-05 order",  
     ylab = "Units", xlab = "Month", pch = 19, xlim = c(0.5, 4.5),  
     ylim = c(50, 350)  
 )  
points(4, counts[4], pch = 3)
```



Making the “printout”

Product 71053: 2011-05 order



- In this (very particular) case, it looks like linear regression would perform quite well ...

SQL injection

- Never use `paste0` directly to construct query strings



—xkcd

-
- Better to use `sqlInterpolate` from the library `DBI`:

```
library(DBI)
query <- sqlInterpolate(db, "SELECT SUM(Quantity) FROM sales
                             WHERE StockCode = '71053' AND
                             InvoiceDate LIKE ?month",
  month = paste0(months[i], "%")
)

request <- dbSendQuery(db, query)
...
```

- This substitutes trailing arguments into the respective question marks in the query string, in a way that doesn't allow injection

Linear Regression

Consulting

“Meet The World’s Best Management Consulting Firms 2024 ... Accenture ... McK-
insey ... Deloitte ...

—Forbes

Thought process of top consulting firms:

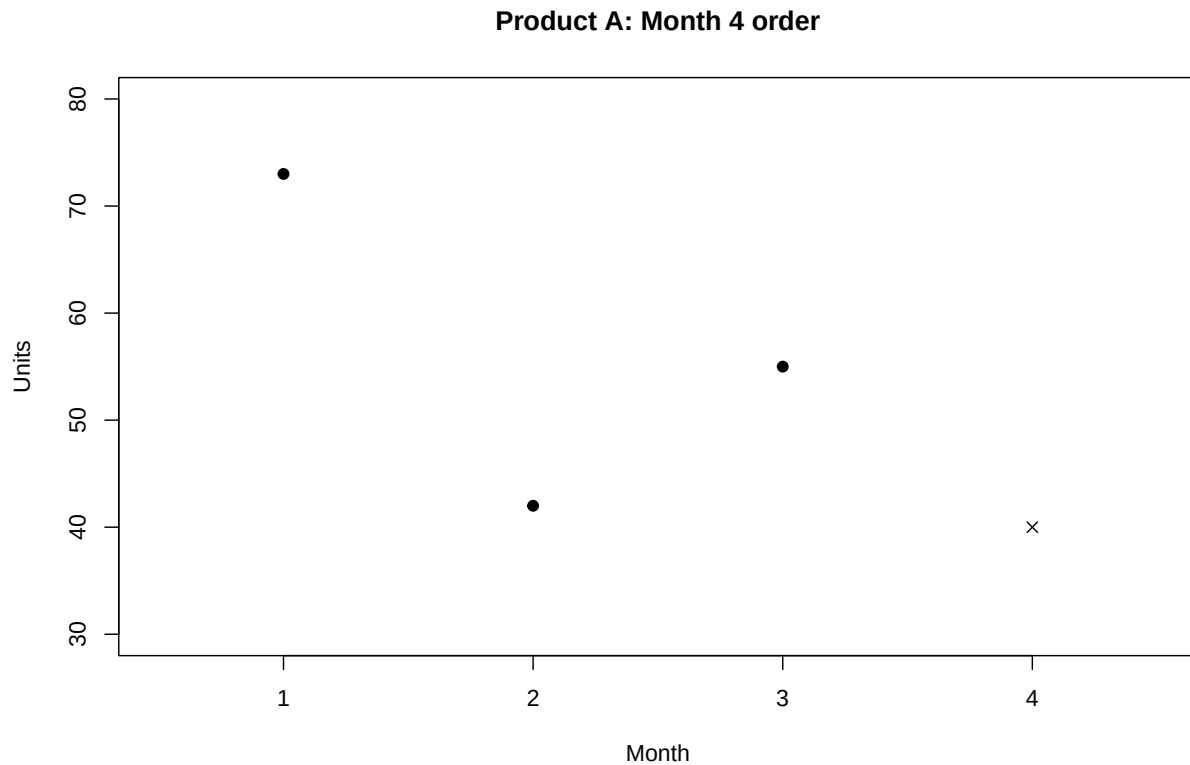
- If the best possible solution for a client is 100%, then linear regression will often take us to 80%
- Linear regression is easy, the tricky part is figuring out what to put into it
- Does 80% bring us a business case (i.e, make us money)?

Consulting

- Being a successful consultant requires solving a variety of different problems, with different data, and communicating those solutions in an understandable way
- Often similar to what a data scientist ends up doing
- Get asked to solve a problem, need to find a suitable solution efficiently
- Hence why linear regression so useful
 - Will see some other key methods in a few weeks

Linear regression: Overview

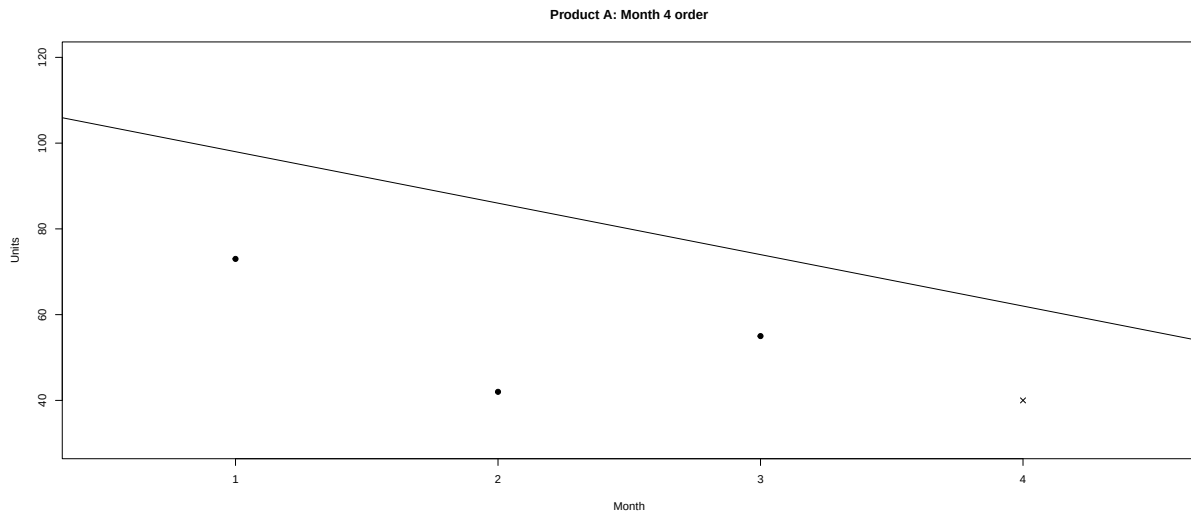
- Consider the “printout” from earlier:



- The cross a guess of what the *Units* value for *month* 4 is
- We would like to be able to predict this well!

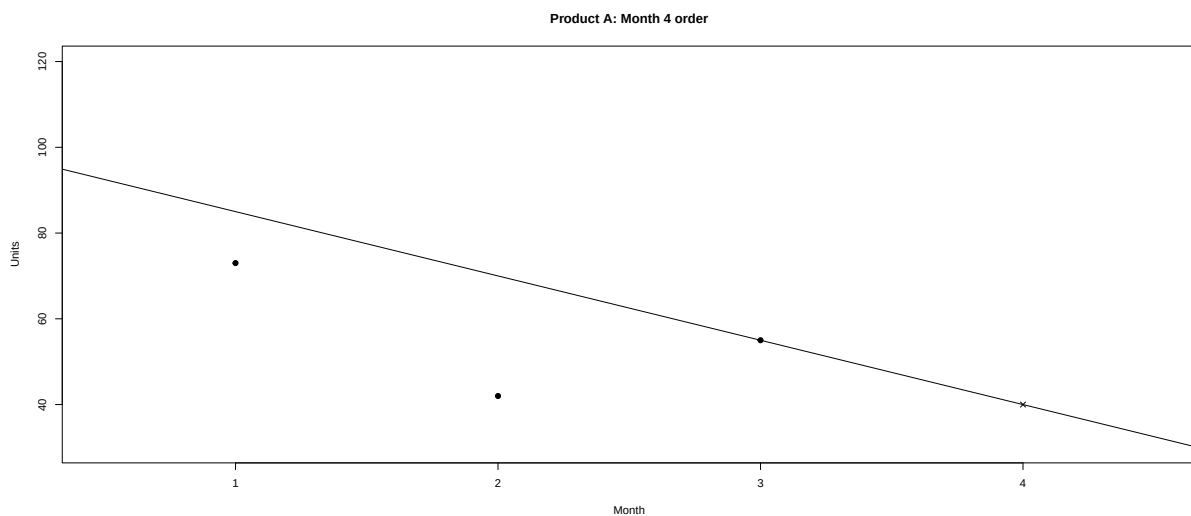
Linear regression

- What if we fit the best line through the points? Would the y value of the line at $x = 4$ be close to the cross? The best fit line wouldn't look like this:



Linear regression

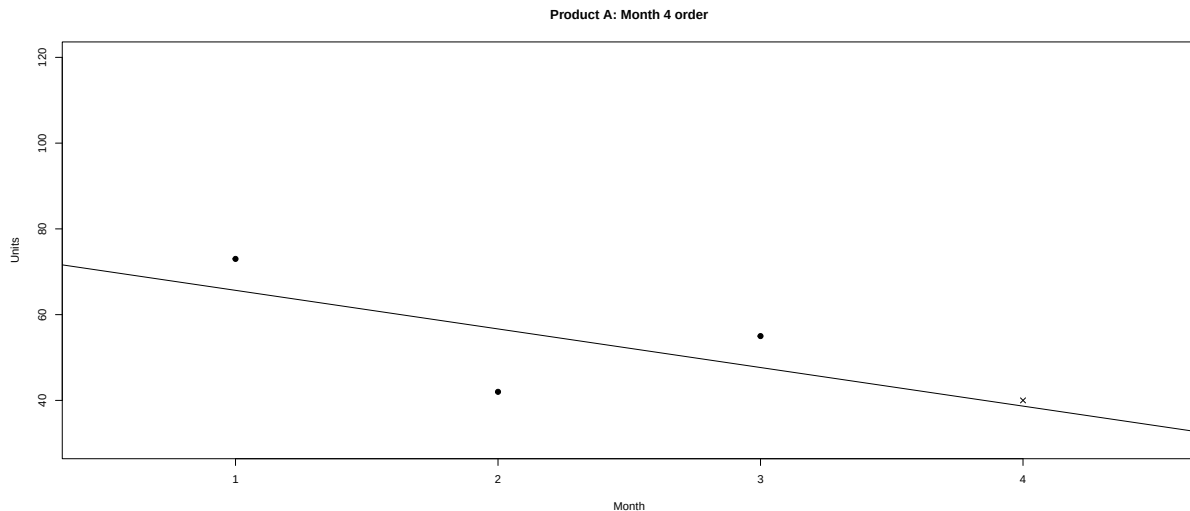
- Can draw a “better” line, this one for example:



- What does better mean here? The distance from the points to the lines is smaller

Linear regression: Solution

- The *ordinary least squares* (OLS) solution to regression is the line that minimizes the sum of squares of the distances between the points and the lines (“better” means “smaller sum of squared errors”).



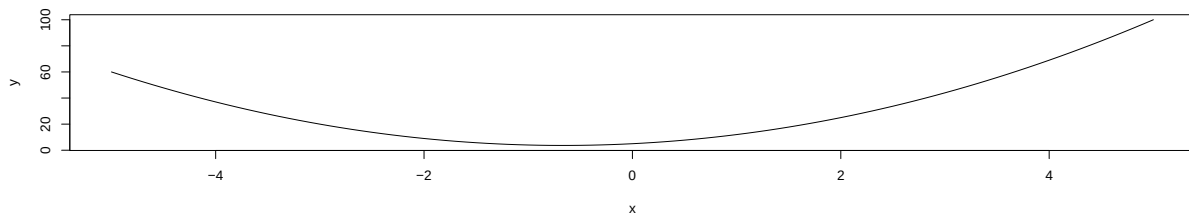
A brief review of optimization

- The solution on the previous slide minimized some quantity
- Many (all?) problems in statistics and data science can be considered as optimization problems
- Want to maximize or minimize some quantity

-
- This is the case for a lot of data science tasks too. You might be asked to find a way to buy the right amount of stock to optimize for:
 - Profit
 - Cost
 - Demand
 - Some arbitrary quantity that doesn't make sense (if your boss isn't that technical)
 - Useful to know some basic ideas in optimization, will probably see them in other courses

Simple Optimization

- You've seen some of this in High School Math
- Suppose you want to minimize $y = 3x^2 + 4x + 5$

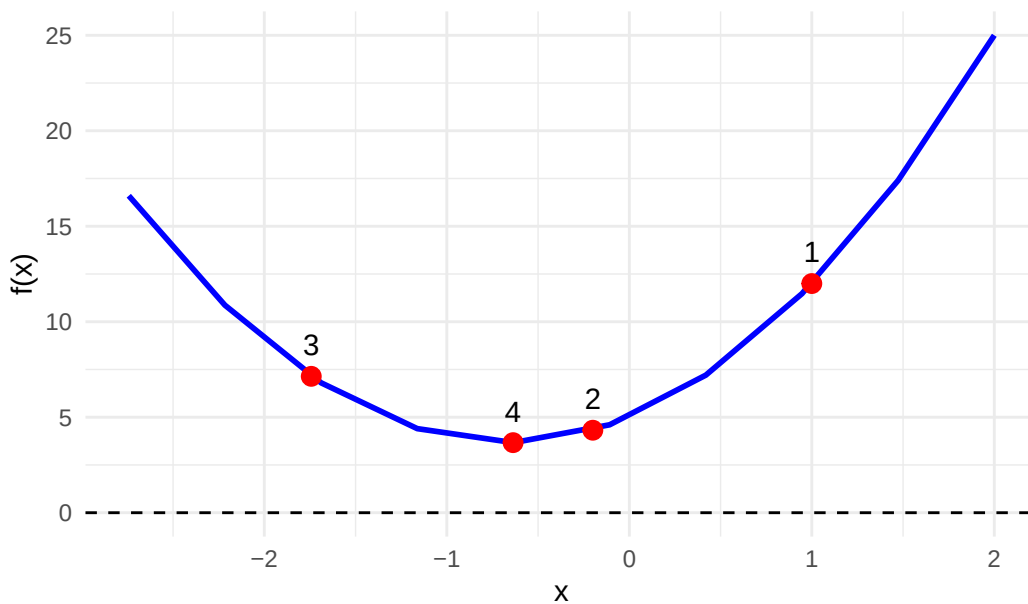


-
- Take the derivative, solve for x when $\frac{dy}{dx} = 0$
 - Because the second derivative is positive, you know this is a global minimum
 - This is because the quantity we want to minimize, $3x^2 + 4x + 5$ is **convex**
 - Our objective in linear regression also convex
 - Convex problems relatively easy to solve

Other optimization methods

- You might have seen a method called Newton's method

Newton's Method for Root Finding



- There are lots of other **iterative** methods like this, which can converge to the solution if you run them for long enough and your problem is not too hard
- We will not talk about them much more here except that you can implement most common methods in R using the function `optim`
- Useful to know how to use this
- Will see how to do this for Linear Regression, but there is a built in function in R we will normally use!

Linear regression: Solution equation

- Suppose $(X_1, Y_1), \dots, (X_n, Y_n)$ are pairs of variables
- Y is the response variable and X is the explanatory variable (X is the variable we have control over, or Y is the variable that we expect is effected by X)
- Our model is the following: $f(X) = \alpha + \beta X$
- We will have randomness about this model, but we can't do much about that
 - Make assumptions that it's nicely behaved

-
- The values of β and α that minimize the sum of squared errors between Y and $f(X)$ are:

$$\hat{\beta} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}, \quad \hat{\alpha} = \bar{Y} - \hat{\beta}\bar{X}. \quad (1)$$

- Here $\bar{X}$ is the mean of $X_1, \dots, X_n$ and $\bar{Y}$ is the mean of $Y_1, \dots, Y_n$. *Discussion:* How would you try to prove this formula?

Linear regression: Code

- Given vectors xs and ys in R, the intercept and slope for the OLS solution can be calculated with the following code:

```
xs <- c(1, 2, 3)
ys <- c(73, 42, 55)
model <- lm(y ~ x, data = data.frame(x = xs, y = ys))
alpha <- summary(model)$coefficients[1, 1]
beta <- summary(model)$coefficients[2, 1]
alpha
```

```
[1] 74.66667
```

```
beta
```

```
[1] -9
```

```
model <- lm(y ~ x, data = data.frame(x = xs, y = ys))
summary(model)
```

Call:

```
lm(formula = y ~ x, data = data.frame(x = xs, y = ys))
```

Residuals:

```
      1      2      3
7.333 -14.667  7.333
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	74.67	27.44	2.721	0.224
x	-9.00	12.70	-0.709	0.608

Residual standard error: 17.96 on 1 degrees of freedom

Multiple R-squared: 0.3343, Adjusted R-squared: -0.3315

F-statistic: 0.5021 on 1 and 1 DF, p-value: 0.6076

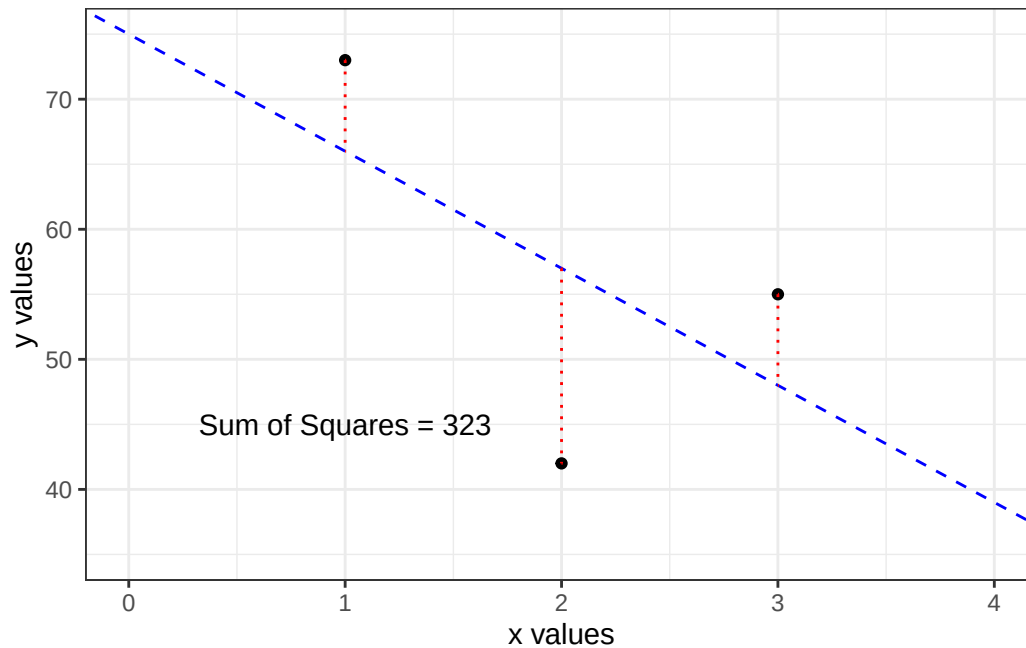
- The function *lm* is a powerful function in R. The acronym stands for *linear model* and the first argument uses R's "formula" notation

Linear regression: Theory

- Why do we care about the sum of squared errors when we fit a line?
- The reason is statistical. Imagine there was a “true” value of α and β and that the Y_i values were generated by the following process:
- Set $Z_i = \alpha + \beta X_i$
- Set $Y_i = Z_i + \epsilon_i$, where $\epsilon_1, \dots, \epsilon_n$ are independent random variables following a normal distribution with mean 0 and standard deviation σ

-
- Under this generation process, the OLS solution $\hat{\alpha}, \hat{\beta}$ is the *maximum likelihood estimate* for α and β
 - In nature, many variables are independent and normally distributed (this is because of the *central limit theorem*)

Sum of Squared Errors



-
- The red vertical distance is distance between the estimated line and the true y value

- Our model is $f(X) = \alpha + \beta X$
- We want to minimize

$$SSE = \sum_{i=1}^n (y_i - f(x_i))^2 = \sum_{i=1}^n (y_i - (\alpha + \beta x_i))^2$$

- We want to choose the pair of values (α, β) which make SSE as small as possible

$$SSE = \sum_{i=1}^n (y_i - f(x_i))^2 = \sum_{i=1}^n (y_i - (\alpha + \beta x_i))^2$$

- Take the derivatives of this with respect to α, β , solve these two equations

$$\hat{\beta} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}, \quad \hat{\alpha} = \bar{Y} - \hat{\beta}\bar{X}. \quad (2)$$

- Because this a convex function ((something)²) this is the best/only solution

Exercise

- Write a function **SSE** which will have four arguments
 - **alpha**
 - **beta**
 - **x**, a vector of x values
 - **y**, a vector of y values
- That will return the **SSE** for given α, β, x, y

Using **optim** in R

- If we can write a function to compute the SSE for values of α, β , we can use **optim** to find the best values


```
SSE <- function(pars = c(0, 0), x, y) {
  alpha <- pars[1]
  beta <- pars[2]
  f_x <- alpha + beta * x
  SSE <- sum((y - f_x)^2)
  SSE
}
```

- Here `x,y` vectors of values, we set default values of `alpha,beta` as elements of `pars`

Check this works

- For a simple case

```
x <- c(0, 0, 0)
y <- c(1, 1, 1)

SSE(pars = c(0, 0), x, y)
```

```
[1] 3
```

```
x <- c(0, 0, 0)
y <- c(1, 1, 1)
SSE(pars = c(1, 0), x, y)
```

```
[1] 0
```

The optim function

```
xs <- c(1, 2, 3)
ys <- c(73, 42, 55)

est_fit <- optim(par = c(0, 0), SSE, x = xs, y = ys)
est_fit$par
```

```
[1] 74.657642 -8.994952
```

```
est_fit$convergence ## 0 means the optimization converged
```

```
[1] 0
```

```
model$coefficients
```

```
(Intercept)          x  
  74.66667    -9.00000
```

- `optim` is very flexible, many other ways to use it
 - See `?optim`

Bonus Exercise

- Use `optim` to find the x that gives the minimum value of $f(x) = 3x^2 + 4x + 5$
- What if you wanted to maximise something?